Scalable Microservices Architecture with ECS and Fargate

Project: Scalable Microservices Architecture with ECS and Fargate

Services: Amazon ECS, AWS Fargate, Amazon ECR, Amazon RDS, Amazon CloudWatch

Description: Containerize a microservices-based application using Docker. Deploy the application using Amazon ECS with AWS Fargate and manage logs using Amazon CloudWatch. Use Amazon RDS as the backend database service.

Skills: Containers, Docker, Microservices, Monitoring, Cloud Deployment

Definition: The Scalable Microservices Architecture with ECS and Fargate project is designed to deploy containerized applications in a scalable and serverless environment using AWS services. The project uses Docker containers stored in Amazon ECR and deployed through Amazon ECS with AWS Fargate. Amazon RDS is used as the backend database, while Amazon CloudWatch monitors application logs and task performance. This architecture eliminates server management and enables scalable cloud-native application deployment.

Scope of the Project: The scope of this project is to design and implement a scalable microservices architecture using AWS container services. The project focuses on deploying Dockerized applications with serverless container management and centralized monitoring.

This project covers:

- Containerized Application Deployment:
  Deploy Docker-based microservices using ECS and Fargate.
- Container Image Management:
  Store and manage Docker images using Amazon ECR.
- Scalable Serverless Infrastructure:
  Run containers without managing servers using AWS Fargate.
- Database Integration:
  Connect application containers with Amazon RDS MySQL database.
- Monitoring and Logging:
  Monitor application logs and container activity using Amazon CloudWatch.

Methodology:

1. ECS + Fargate + ECR – Best Method

- Uses all required AWS services: ECS, Fargate, ECR, RDS, and CloudWatch.
- Fully serverless container deployment.
- No EC2 server management required.
- Scalable and production-ready architecture.
- Centralized logging using CloudWatch.
- Cost-effective for small workloads and student projects.
- Simplifies container orchestration and deployment.

2. EC2 + Docker – Alternative Method

- Deploy Docker containers directly on EC2 instances.
- Requires manual server configuration and maintenance.

- Scaling and monitoring need additional setup.
- Higher operational overhead compared to Fargate.
- Suitable for traditional container deployments.

AWS Services used in the Project:

1. Amazon ECS (Elastic Container Service)

Definition: Amazon ECS is a fully managed container orchestration service that helps deploy and manage Docker containers.

Purpose: Manage and orchestrate containerized applications.

Role:

- Deploys microservice containers.
- Manages task execution and service availability.

2. AWS Fargate

Definition: AWS Fargate is a serverless compute engine for containers.

Purpose: Run containers without managing servers.

Role:

- Executes ECS tasks.
- Automatically manages infrastructure resources.

3. Amazon ECR (Elastic Container Registry)

Definition: Amazon ECR is a fully managed Docker container registry service.

Purpose: Store and manage Docker images securely.

Role:

- Stores Docker container images.
- Provides image access to ECS tasks.

4. Amazon RDS (Relational Database Service)

Definition: Amazon RDS is a managed relational database service.

Purpose: Provide backend database connectivity for applications.

Role:

- Stores application data.
- Provides MySQL database support.

5. Amazon CloudWatch

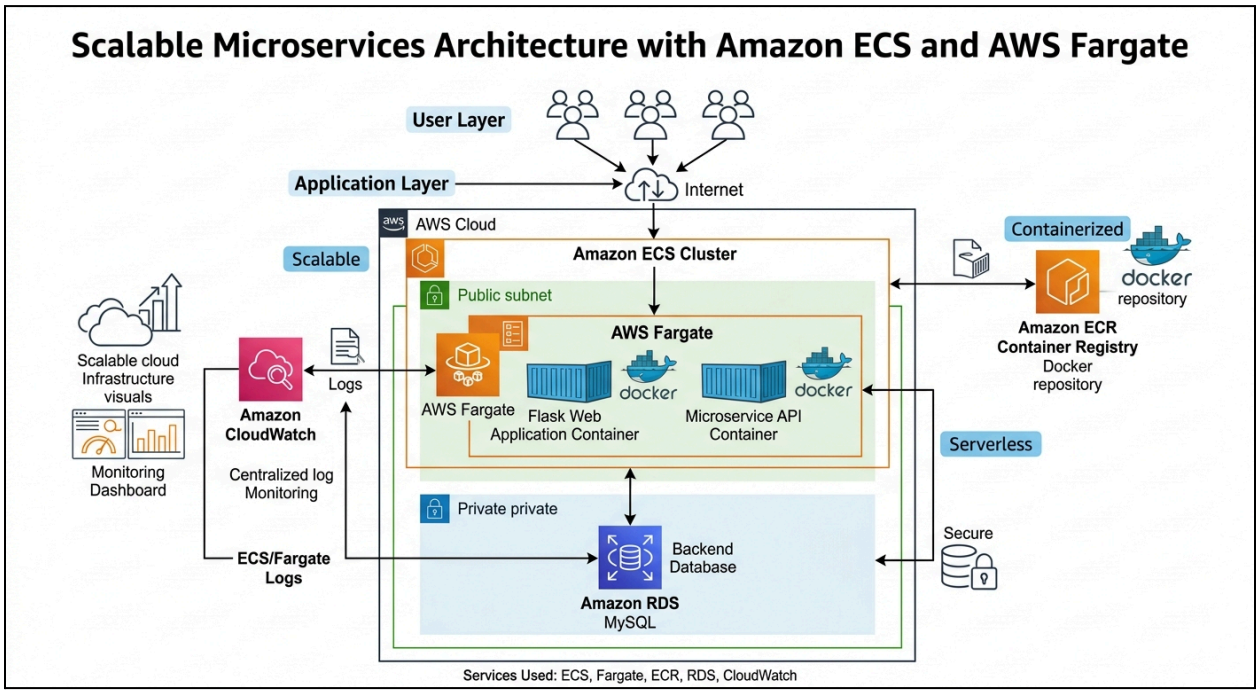
Definition: Amazon CloudWatch is a monitoring and logging service for AWS resources and applications.

Purpose: Monitor logs and application performance.

Role:

- Collects ECS container logs.
- Monitors application activity and errors.

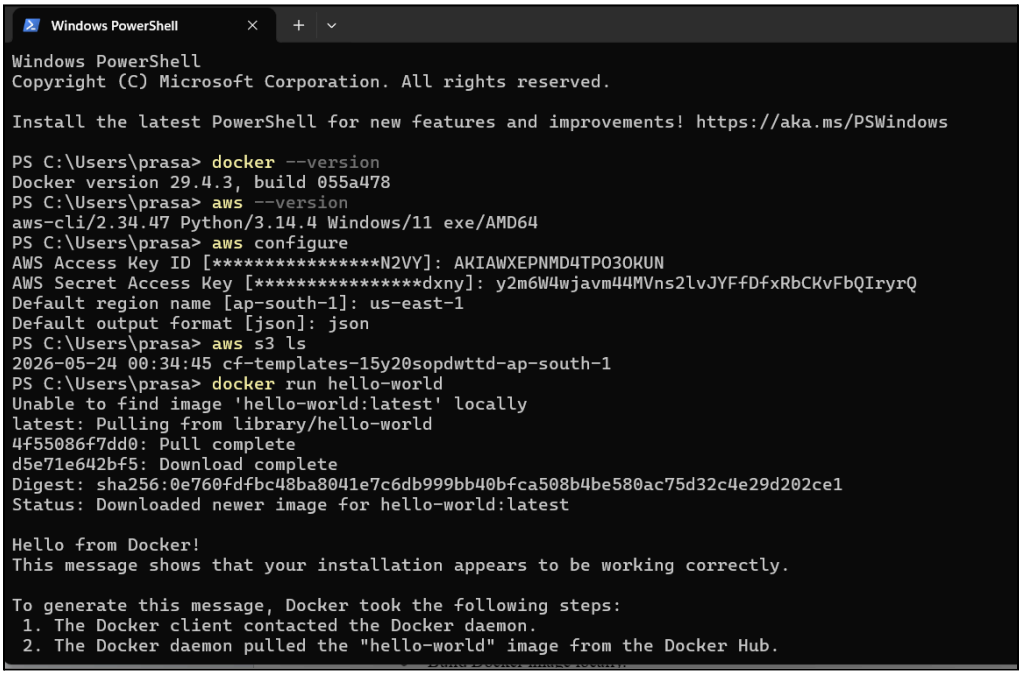
Architecture Diagram:



Implementation Steps to Do the Project:

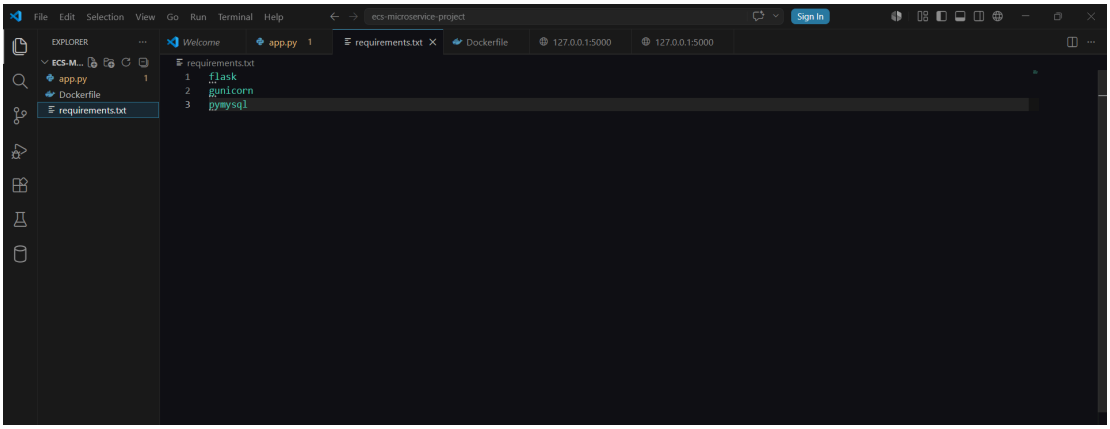
Step 1: Install Docker and AWS CLI

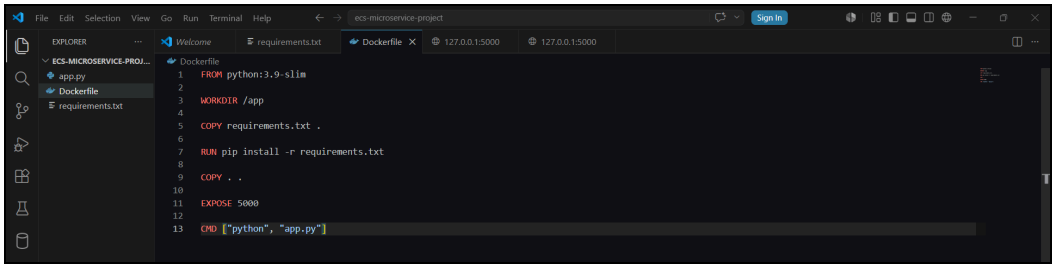
- Install Docker Desktop on local system.
- Install AWS CLI.
- Configure AWS credentials using aws configure.

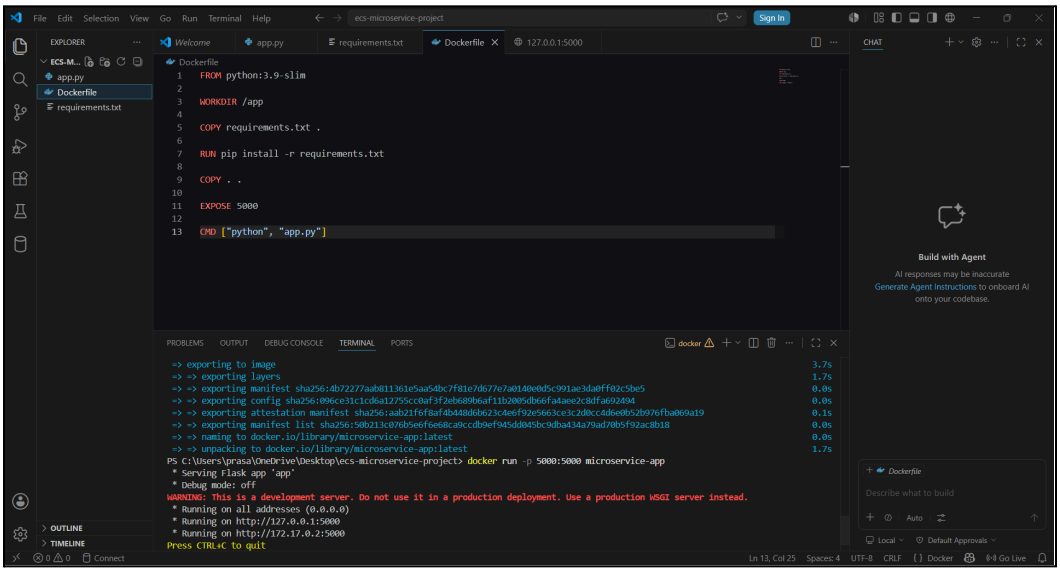


Step 2: Create Dockerized Application

- Create Flask microservice application.
- Create requirements.txt.
- Create Dockerfile.
- Build Docker image locally.

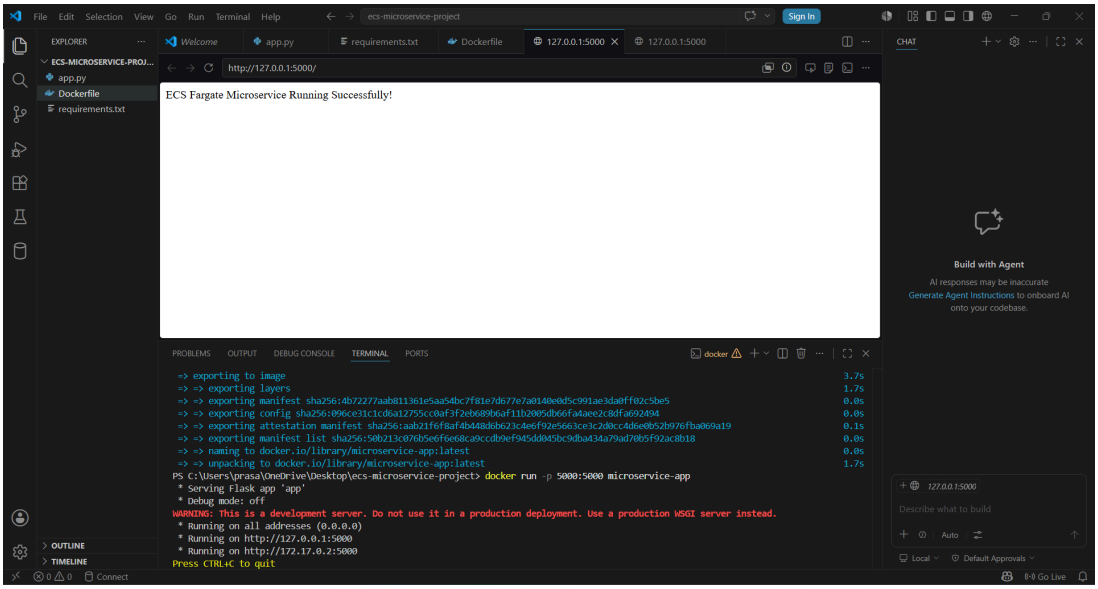






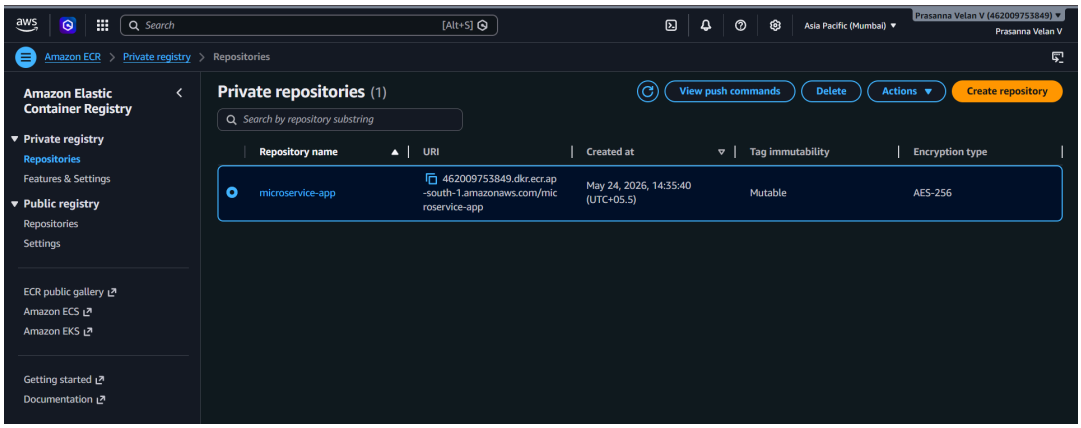
Step 3: Test Docker Container Locally

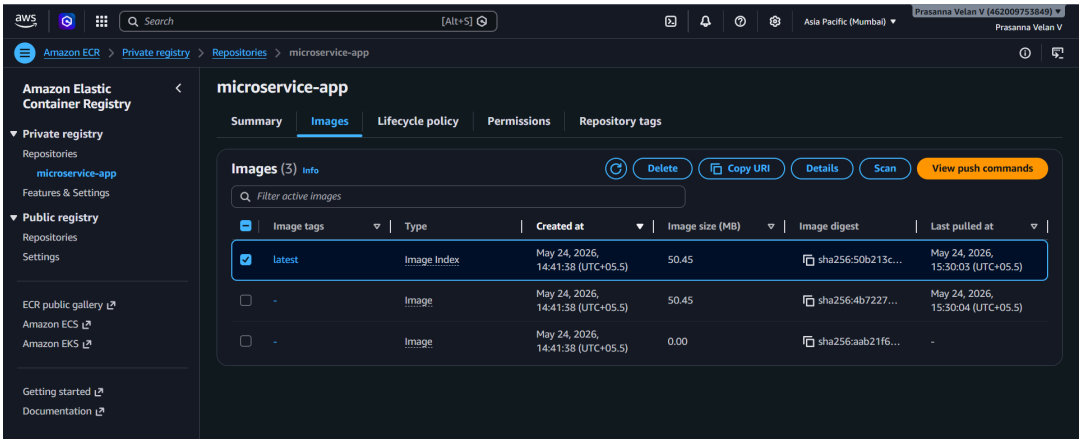
- Run Docker container locally.
- Verify application using localhost port 5000.



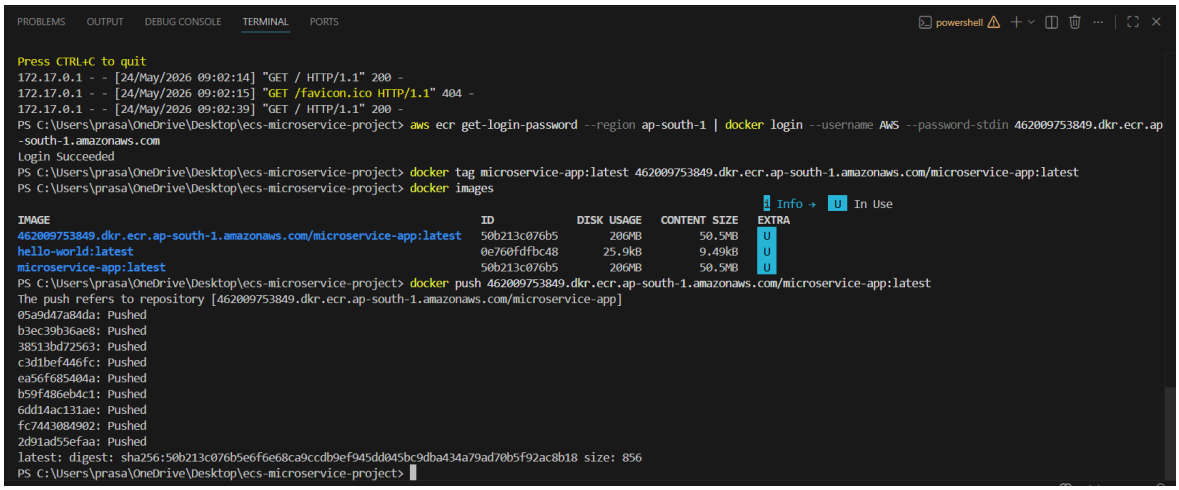
Step 4: Create Amazon ECR Repository

- Go to Amazon ECR.
- Create private repository named microservice-app.
- Authenticate Docker with ECR.
- Push Docker image to ECR.



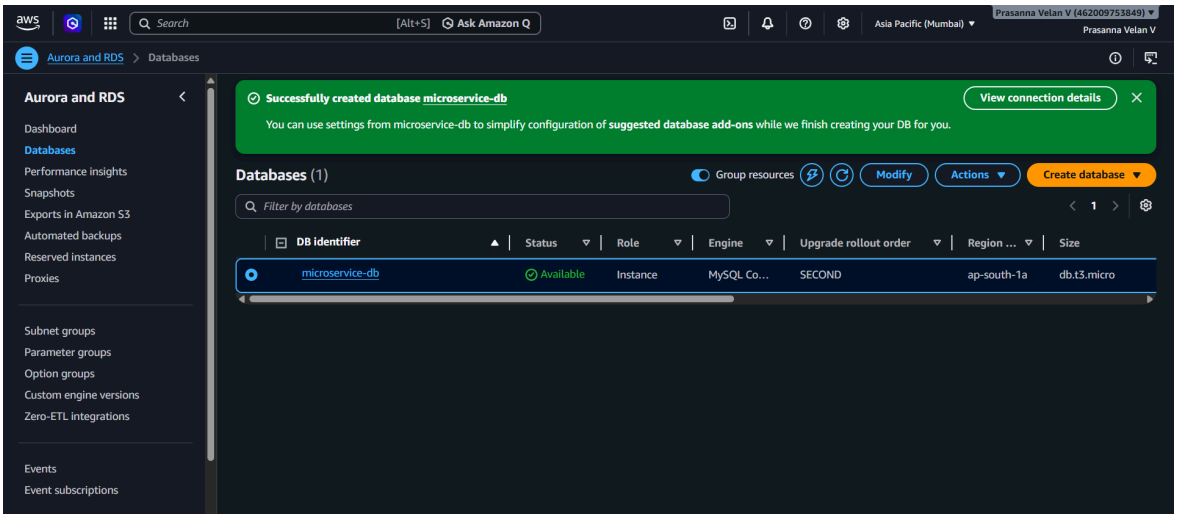


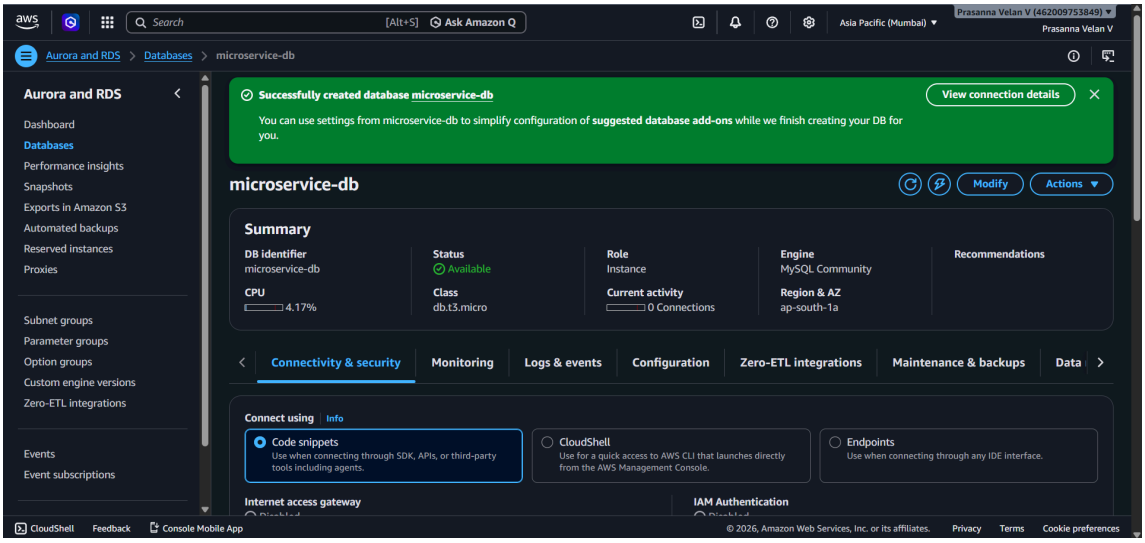

```
PS C:\Users\prasa\OneDrive\Desktop\ecs-microservice-project> docker run -p 5000:5000 microservice-app
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [24/May/2026 09:02:14] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [24/May/2026 09:02:15] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [24/May/2026 09:02:39] "GET / HTTP/1.1" 200 -
PS C:\Users\prasa\OneDrive\Desktop\ecs-microservice-project> aws ecr get-login-password --region ap-south-1 | docker login --username AW
S --password-stdin 462009753849.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
PS C:\Users\prasa\OneDrive\Desktop\ecs-microservice-project>
```



Step 5: Create Amazon RDS Database

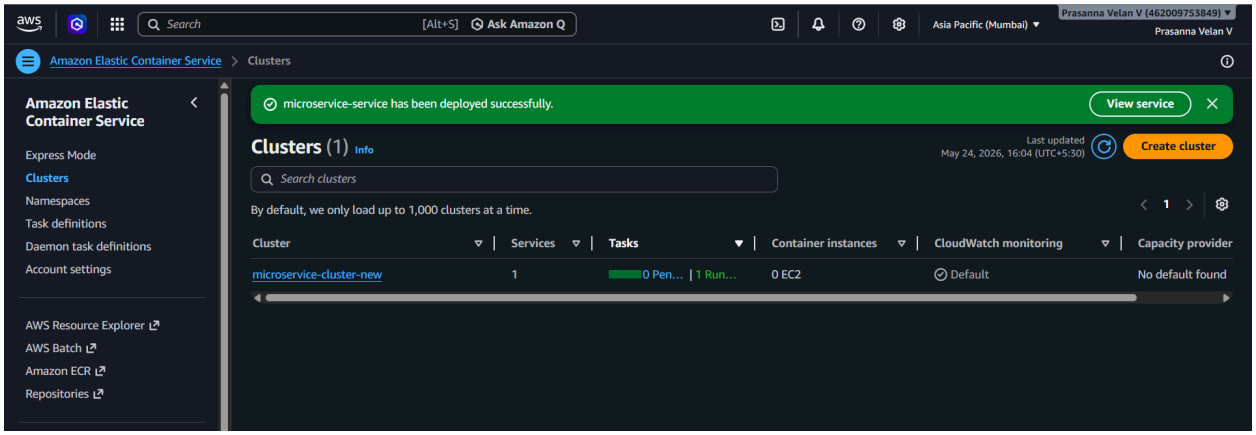
- Open Amazon RDS console.
- Select MySQL database.
- Choose Free Tier template.
- Create database instance.
- Configure username and password.





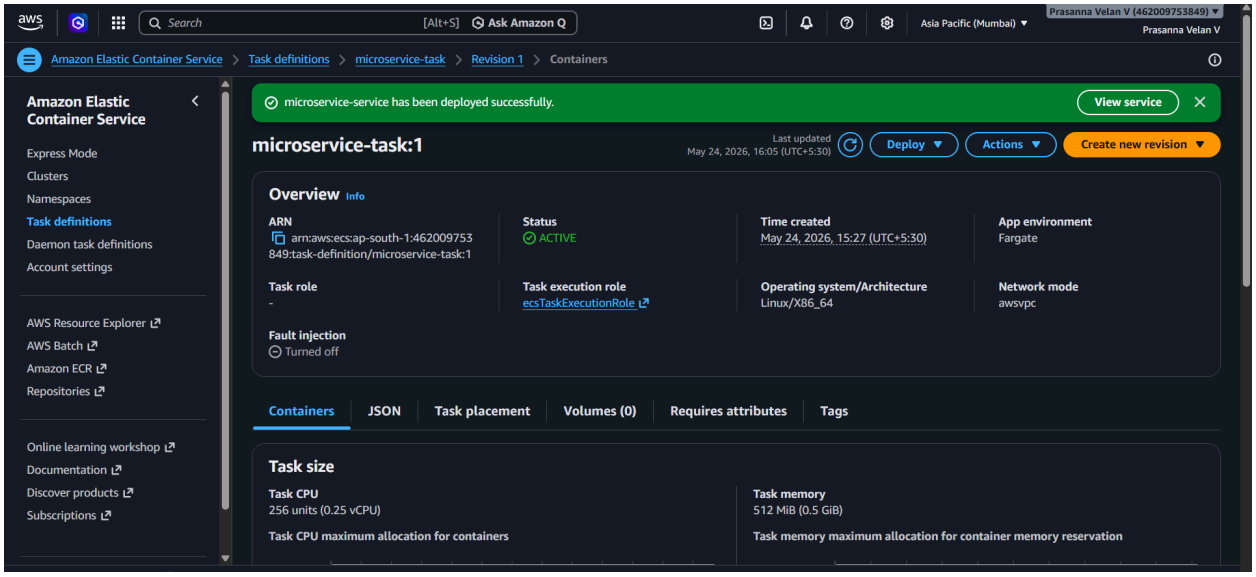
Step 6: Create ECS Cluster

- Open Amazon ECS.
- Create cluster using AWS Fargate.
- Configure cluster name and settings.



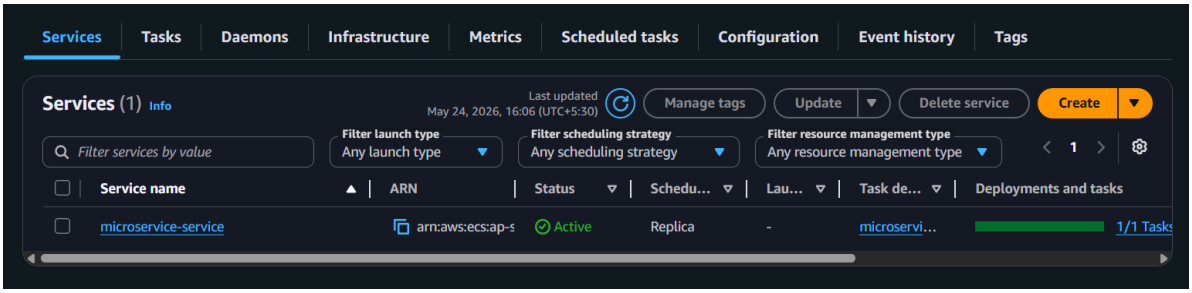
Step 7: Create ECS Task Definition

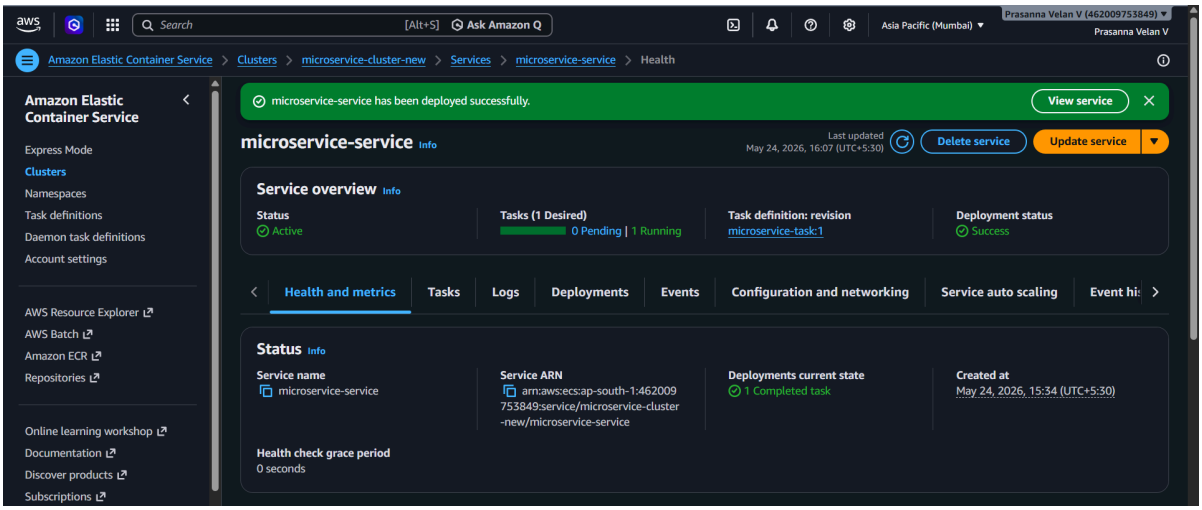
- Create Fargate task definition.
- Add ECR image URI.
- Configure container port mapping.
- Enable CloudWatch logging.



Step 8: Create ECS Service

- Deploy ECS service using task definition.
- Enable public IP.
- Configure networking and security groups.
- Run Fargate task.



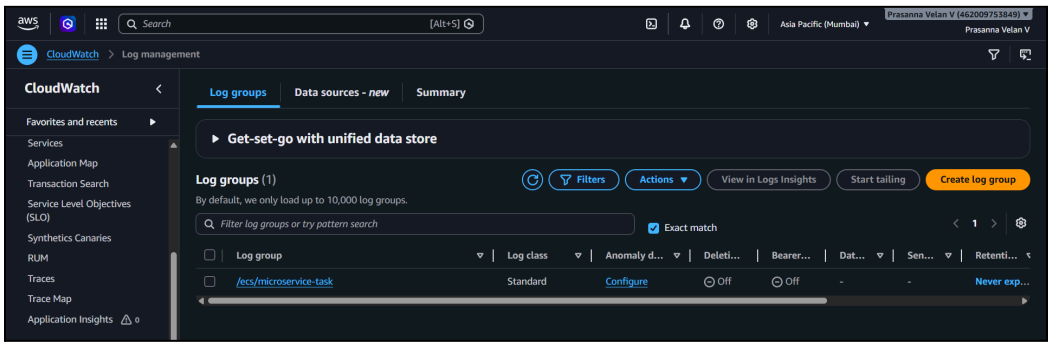


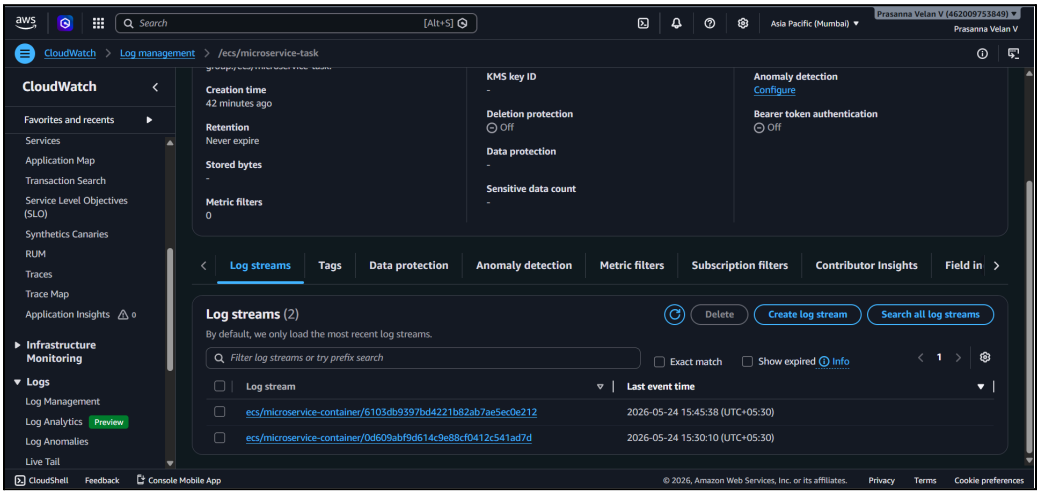
Step 9: Verify Application Deployment

- Open public IP in browser.
- Verify Flask application is running successfully.

Step 10: Verify CloudWatch Logs

- Open CloudWatch Log Groups.
- Verify ECS container logs.
- Monitor application activity.





Troubleshooting:
Issue 1: Docker Build Failed

Docker image build failed due to incorrect Dockerfile or missing dependencies.

Solution:

- Verify Dockerfile syntax.
- Install required Python packages.
- Rebuild Docker image.

Issue 2: ECS Task Not Running

ECS task failed because of incorrect container configuration or networking issues.

Solution:

- Verify ECR image URI.
- Check container port mapping.
- Ensure security group allows port 5000.
- Verify public IP assignment is enabled.

Issue 3: Application Not Accessible

Application page did not open in browser.

Solution:

- Verify ECS task is running.
- Check public IP address.
- Allow inbound rule for TCP port 5000.

Conclusion:

The Scalable Microservices Architecture with ECS and Fargate project successfully demonstrates deployment of containerized applications using AWS cloud services. Docker containers are stored in Amazon ECR and deployed using Amazon ECS with AWS Fargate, eliminating the need for server management. Amazon RDS provides reliable backend database support, while Amazon CloudWatch enables centralized monitoring and logging.

This architecture is scalable, serverless, cost-effective, and suitable for modern cloud-native applications and organizational deployments.